Dynamic Programming

Paradigms

A **greedy algorithm** always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.

A **divide-and-conquer** algorithm breaks the problem into subproblems; solves them; and combines the solutions to obtain a solution to the original problem.

A **dynamic programming** algorithm breaks the problem into a series of overlapping subproblems; combine solutions of smaller subproblems to obtain a solution to the original problem.

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

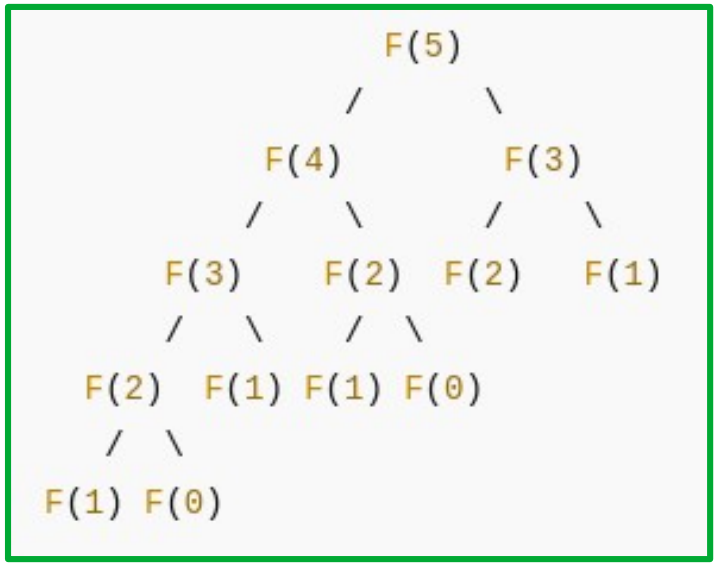
Recursion Tree?

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

Recursion tree



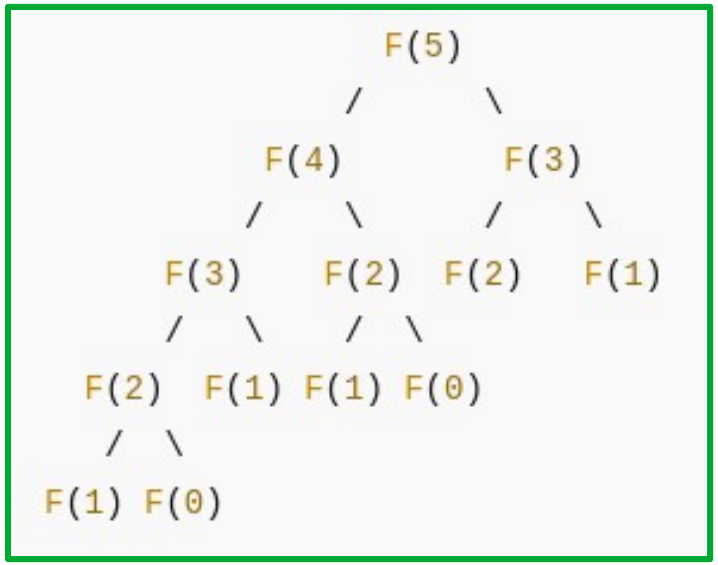

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

Recursion tree




```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

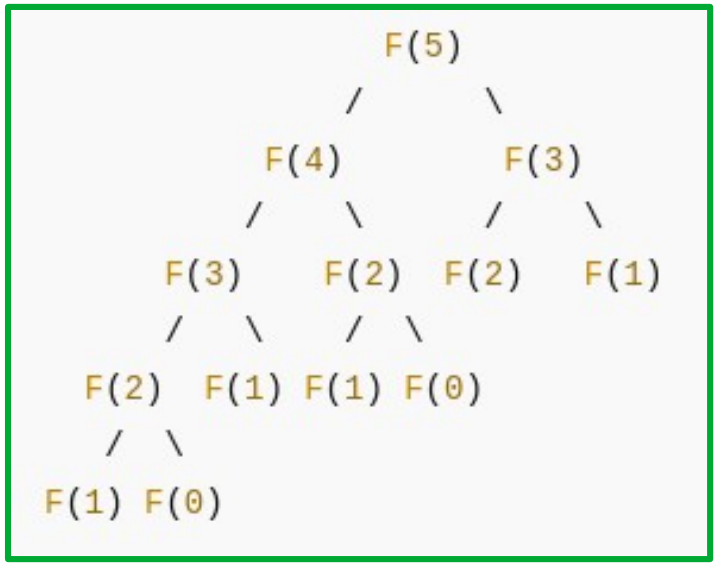
What is the running time of this algorithm?

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

Recursion tree




```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

What is the running time of this algorithm?

$O(2^n)$

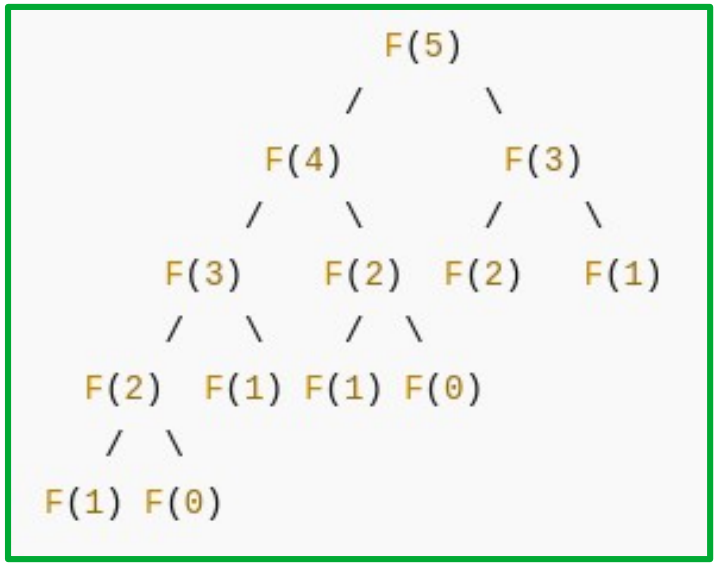
Can we do something to improve the running time?

Problem

Find the nth term of the series defined by

$$F(0)=0,\ F(1)=1,\ \text{and}\ F(n)=F(n-1)+F(n-2)\ \text{for all}\ n \geq 2.$$

Recursion tree




```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

What is the running time of this algorithm?

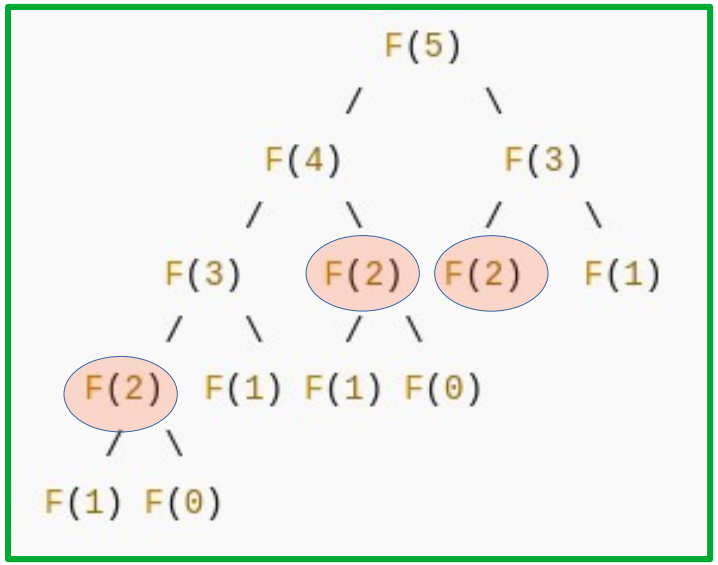
$O(2^n)$ -> $O(1.618^n)$

Problem

Find the nth term of the series defined by

F(0)=0, F(1)=1, and F(n)=F(n-1)+F(n-2) for all n ≥ 2.

Recursion tree



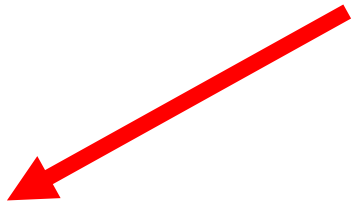

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Approach

Solve each sub-problem once and reuse it.

```
memo = array of size n+1 initialized with -1

fib(n):
    if n <= 1: return n
    if memo[n] != -1: return memo[n]
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```



```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Time: **O(n)**

Space: **Recursion stack + Table**

Approach

Solve each sub-problem once and reuse it.

"**Dynamic**": process of building solutions progressively
"**Programming**": storing values in the form of tables

Top-down DP (Memoization)

```
memo = array of size n+1 initialized with -1

fib(n):
    if n <= 1: return n
    if memo[n] != -1: return memo[n]
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Time: **O(n)**

Space: **Recursion stack + Table**

Approach

Solve each sub-problem once and reuse it.

Features:

NO recursion
Iterative procedure
Space: O(n)

Bottom-up DP (Tabulation)

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Approach

Solve each sub-problem once and reuse it.

Features:

NO recursion
Iterative procedure
Space: O(n)

Bottom-up DP (Tabulation)

```
dp[0] = 0
dp[1] = 1
for i = 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
return dp[n]
```

```
fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

Can we do better?

Approach

Solve each sub-problem once and reuse it.

Features:

NO recursion
Iterative procedure
Space: O(1)

DP ≠ always BIG TABLES

Bottom-up DP (Tabulation)

```
dp[0] = 0
dp[1] = 1
for i = 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
return dp[n]
```



```
prev2 = 0
prev1 = 1
for i = 2 to n:
    curr = prev1 + prev2
    prev2 = prev1
    prev1 = curr
return prev1
```

Dynamic Programming (summary)

- Instead of solving the same subproblems repeatedly, arrange to solve each subproblem just once.
- Save the solution to a subproblem in a table, and refer back to the table whenever we revisit the subproblem.
- “Store, don’t recompute” => time-memory trade-off.
- Can turn an exponential-time algorithm into a **polynomial-time algorithm**.

Two basic approaches: **top-down with memoization**, and **bottom-up**.

Applications

Computer Science & Algorithms

Shortest Path Problems, Knapsack Problem, String Matching:

Artificial Intelligence / Machine Learning

Reinforcement Learning, Markov Decision Processes (MDPs), Hidden Markov Models (HMM),

Bioinformatics

DNA/Protein Sequence Alignment, Gene Prediction and RNA Folding

Operations Research

Job Scheduling, Project Planning, Inventory Management

Economics and Finance

Portfolio Optimization, Optimal Investment and Consumption over Time

Robotics and Control Systems

Game Theory and Competitive Programming

Dynamic Programming

We typically apply dynamic programming to optimization problems.

4 Steps to follow:

1) Characterize the structure of an optimal solution.

Understand **how the best solution to the full problem is built** from the best solutions to smaller subproblems.

Dynamic Programming

We typically apply dynamic programming to optimization problems.

4 Steps to follow:

1) Characterize the structure of an optimal solution.
2) Recursively define the value of an optimal solution.

Write a recurrence relation that **expresses the solution to the full problem** in terms of its subproblems.

Dynamic Programming

We typically apply dynamic programming to optimization problems.

4 Steps to follow:

1) Characterize the structure of an optimal solution.
2) Recursively define the value of an optimal solution.
3) Compute the value of an optimal solution, typically in a bottom-up fashion.

actually **solve the problem** using code, based on your recurrence.

Dynamic Programming

We typically apply dynamic programming to optimization problems.

4 Steps to follow:

1) Characterize the structure of an optimal solution.
2) Recursively define the value of an optimal solution.
3) Compute the value of an optimal solution, typically in a bottom-up fashion.
4) Construct an optimal solution from computed information.

reconstruct the actual solution, not just the final value.

Interval Scheduling

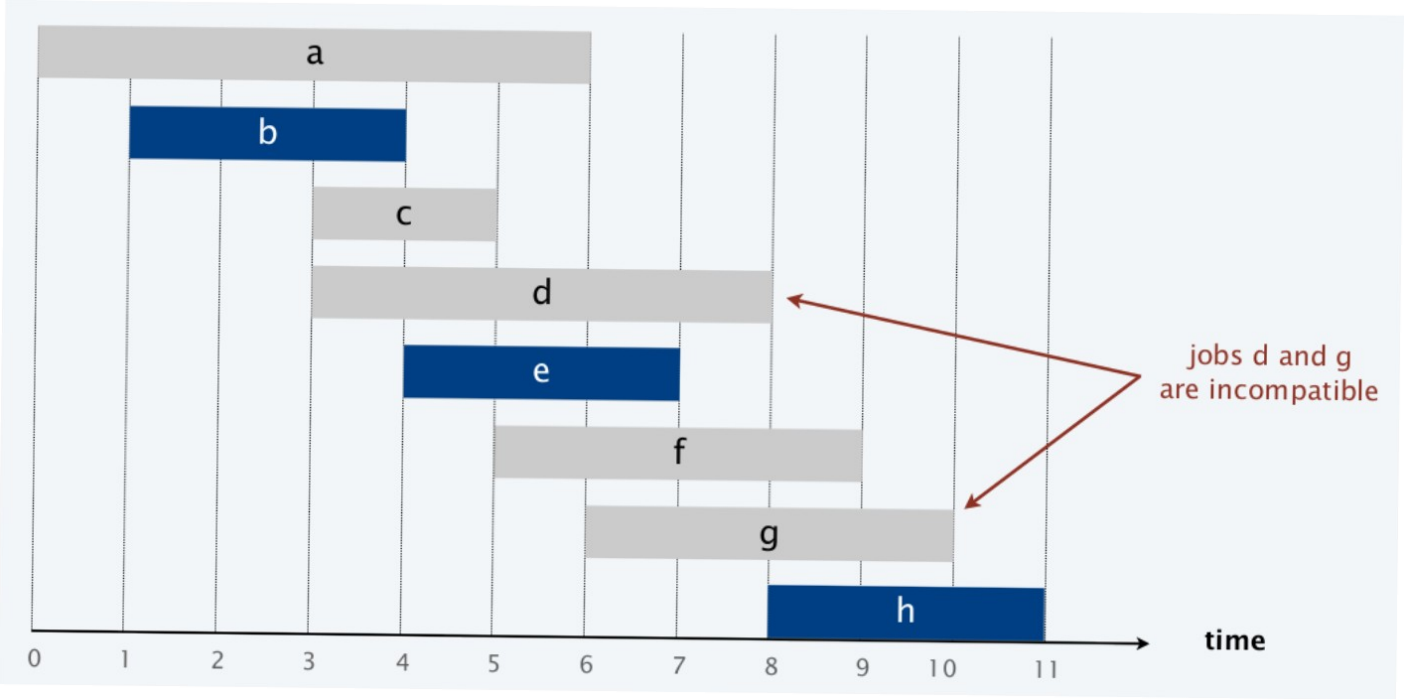
Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.

Interval Scheduling

Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.



Interval Scheduling (algorithms)

Approach	Time
Brute Force	$O(2^n \cdot n^c)$
Greedy	O(n log n) (pick the interval that finishes first)

Weighted Interval Scheduling

Question: Suppose that each job also has a positive weight and the goal is to find a maximum weight subset of mutually compatible intervals. Is the **earliest-finish-time-first** algorithm still **optimal**?

A. Yes, because greedy algorithms are always optimal.

B. Yes, because the same proof of correctness is valid.

C. No, because the same proof of correctness is no longer valid.

D. No, because you could assign a huge weight to a job that overlaps the job with the earliest finish time.

Weighted Interval Scheduling

Input: A set of jobs with start and finish times, with each job assigned a weight;

Question: find the maximum weighted subset of mutually compatible (non-overlapping) jobs.

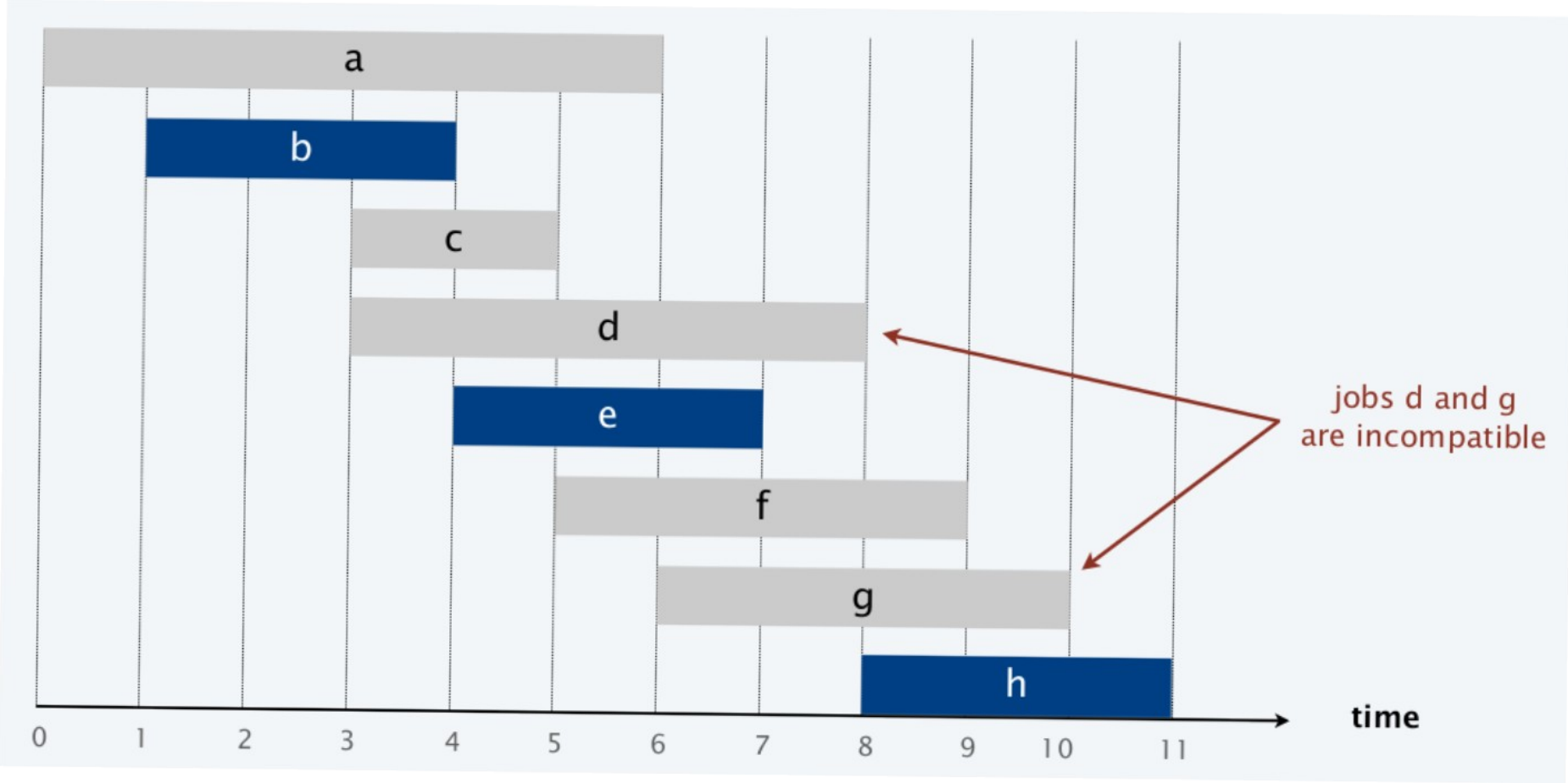
Does the greedy approach work?

Weighted Interval Scheduling (algorithms)

Approach	Time
Brute Force	$O(2^n \cdot n^c)$
Greedy	Does not work
Recursive algorithm	

D&C ?

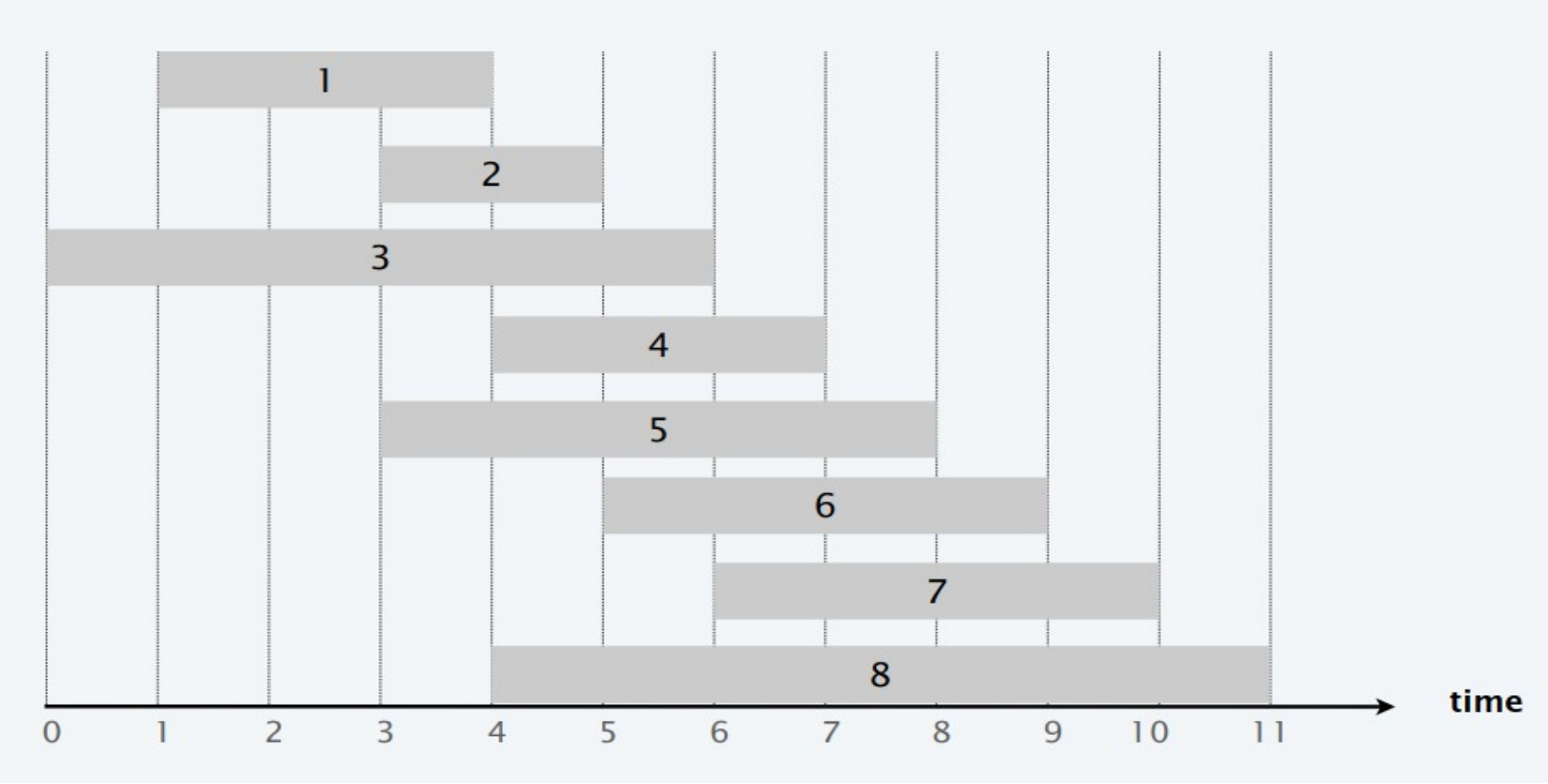
Weighted Interval Scheduling



Weighted Interval Scheduling

1. Sort the Intervals based on their finish times.

Weighted Interval Scheduling

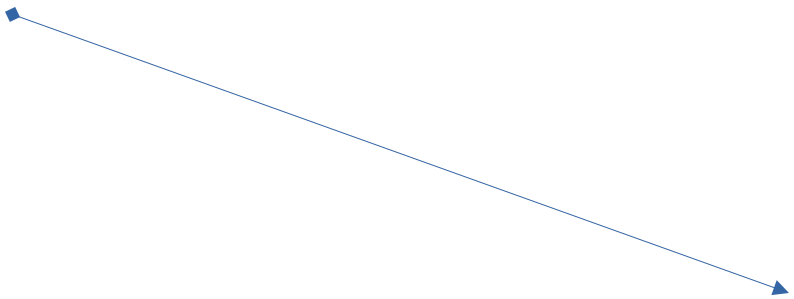


Weighted Interval Scheduling

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j

Weighted Interval Scheduling

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. Return COMPUTE-OPT(n).



Take it or leave it!

Weighted Interval Scheduling

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. Return COMPUTE-OPT(n).

```
COMPUTE-OPT(j)

IF (j = 0)
    RETURN 0.
ELSE
    RETURN max {COMPUTE-OPT(j − 1), w_j + COMPUTE-OPT(p[j]) }.
```

Running time: Cost of each step?

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. Return COMPUTE-OPT(n).

COMPUTE-OPT(j)

IF ($j = 0$)

  RETURN 0.

ELSE

  RETURN max {COMPUTE-OPT($j-1$), w_j + COMPUTE-OPT($p[j]$) }.

Running time: Cost of each step?

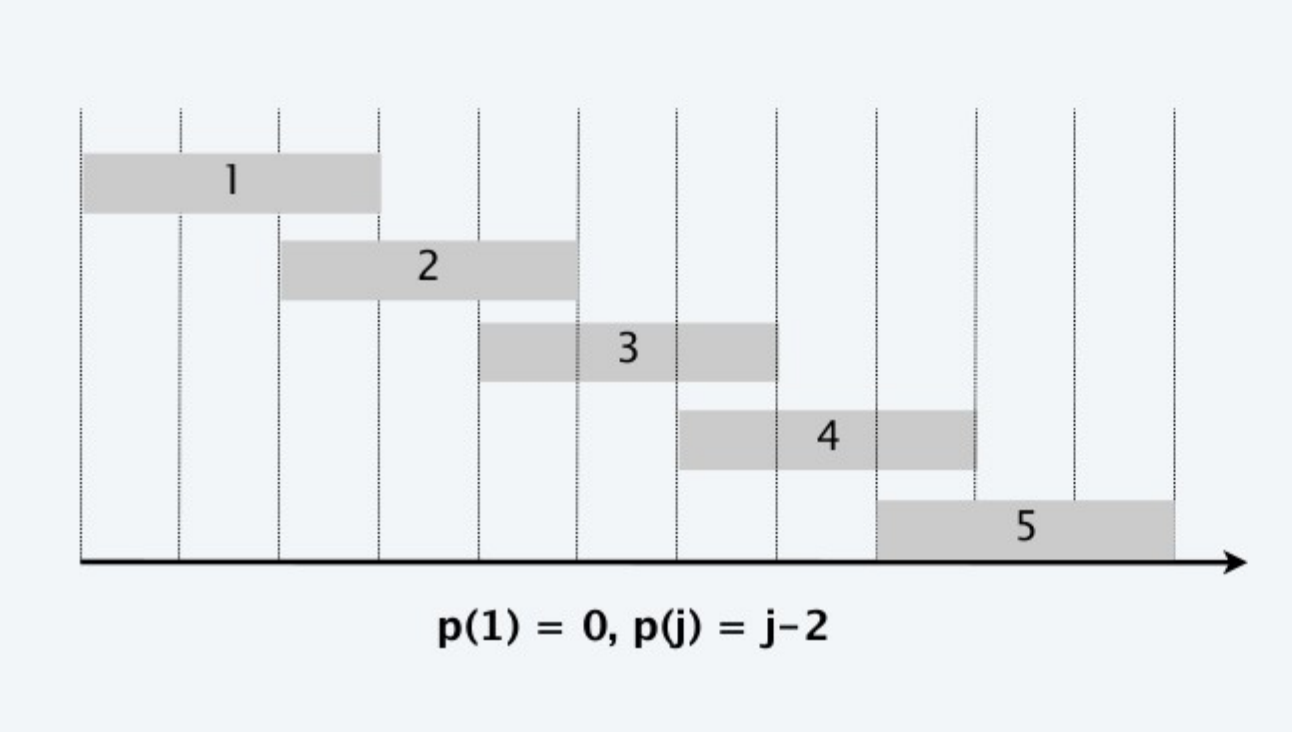
1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. Return COMPUTE-OPT(n).

A. $O(n^2)$
B. O(n log n)
C. $O(1.618^n)$
D. $O(2^n)$

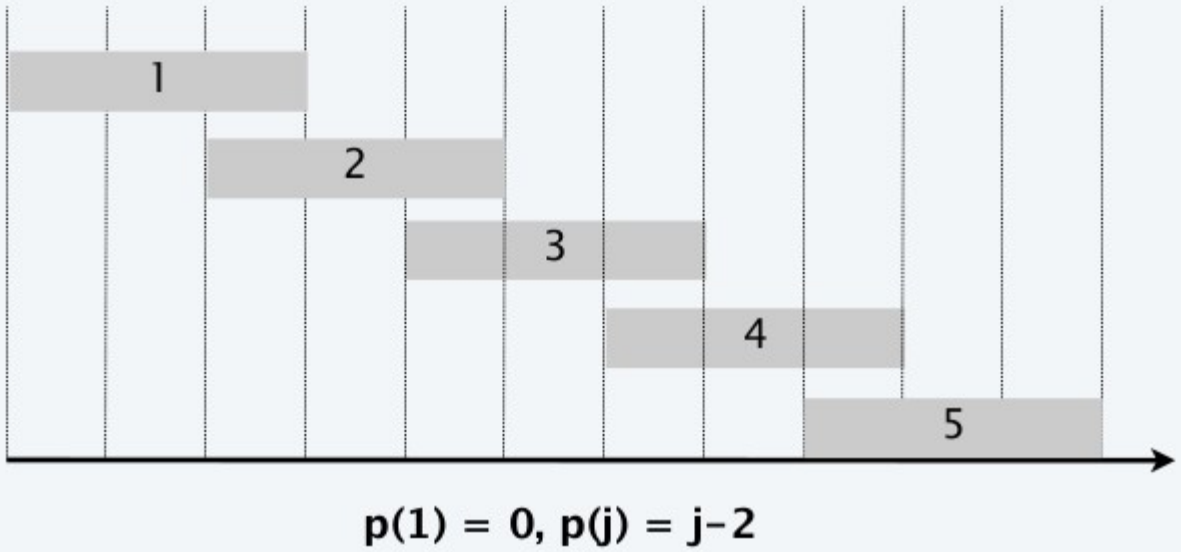
```
COMPUTE-OPT(j)

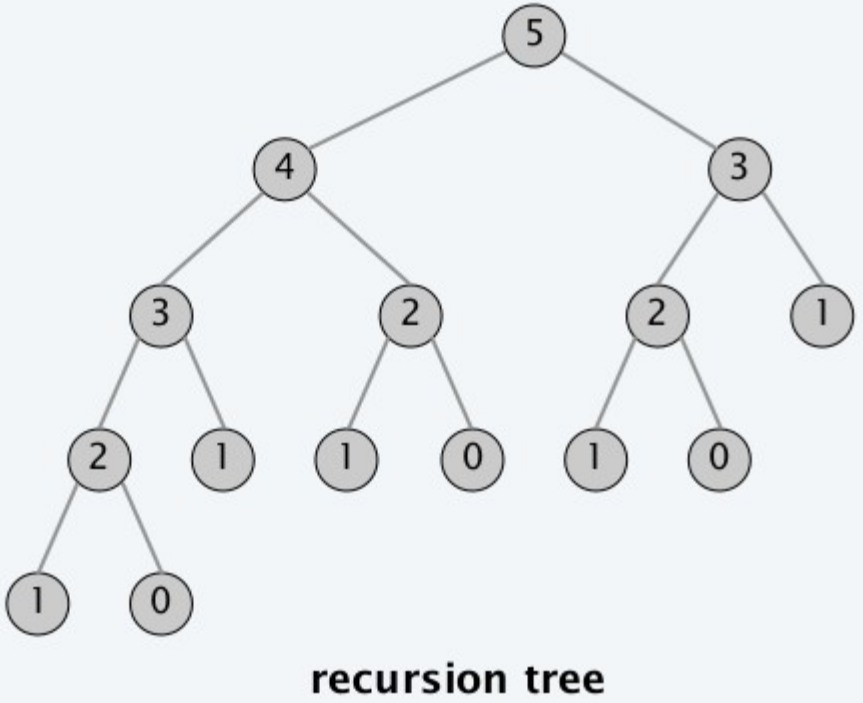
IF (j = 0)
    RETURN 0.
ELSE
    RETURN max {COMPUTE-OPT(j – 1), w_j + COMPUTE-OPT(p[j]) }.
```

Recursive tree

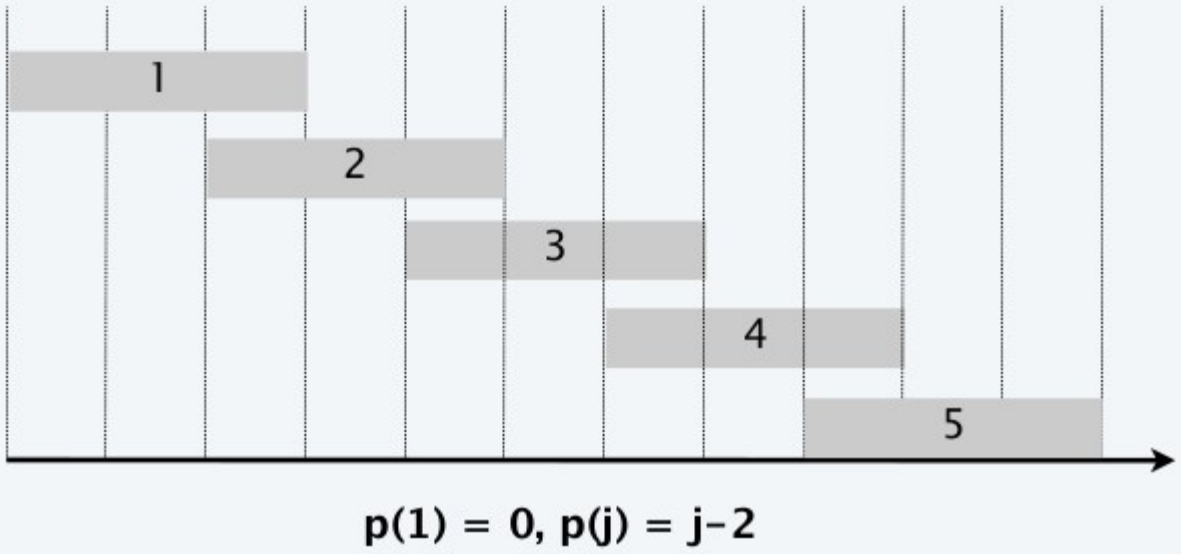


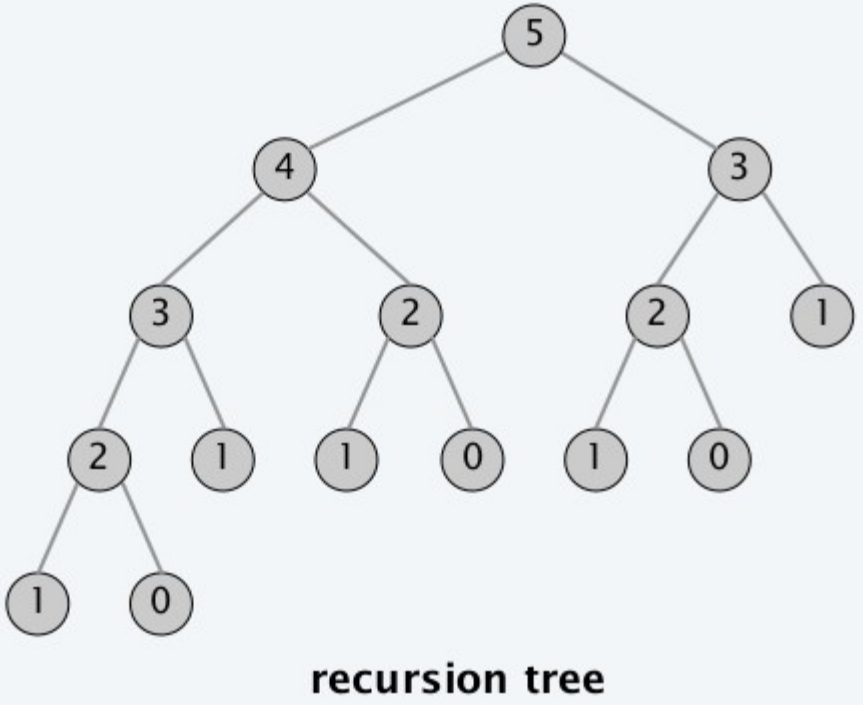
Recursive tree





Question: Running time?

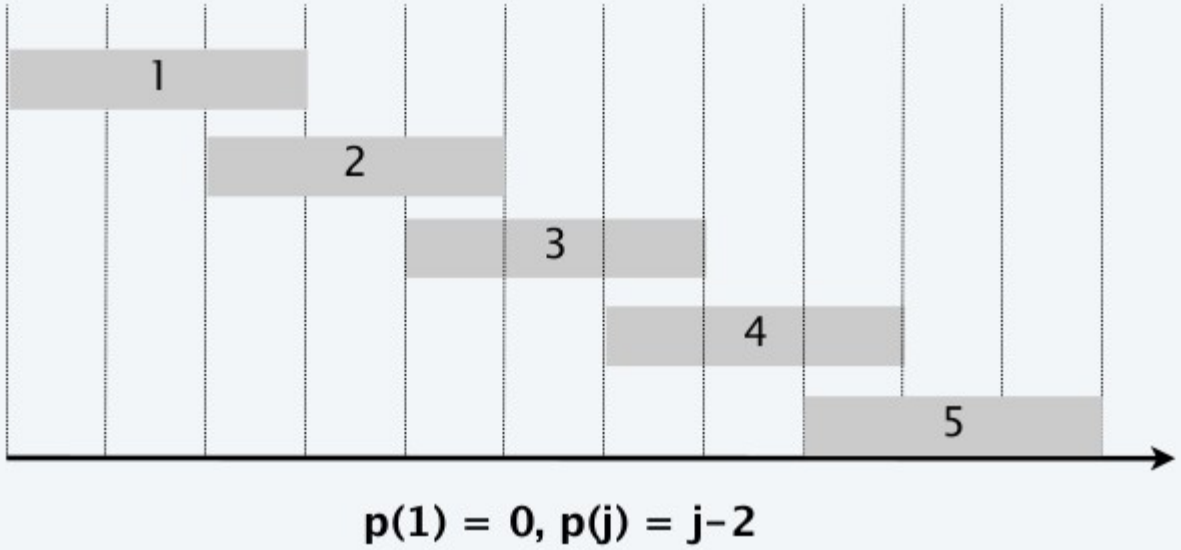


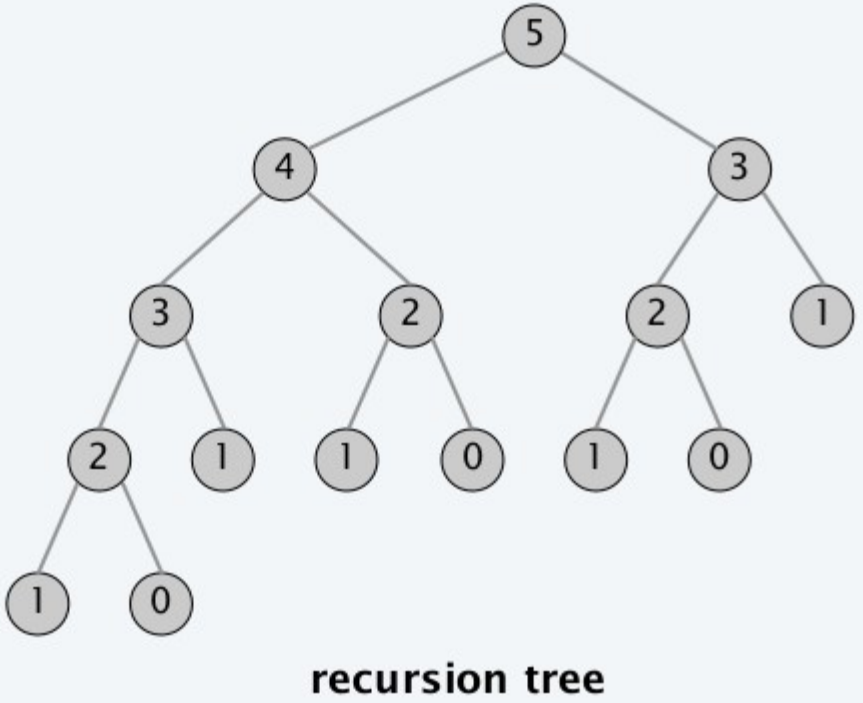


Do you recall this series?

Question: Running time?

A. $O(n^2)$
B. $O(n \log n)$
C. $O(1.618^n)$
D. $O(2^n)$





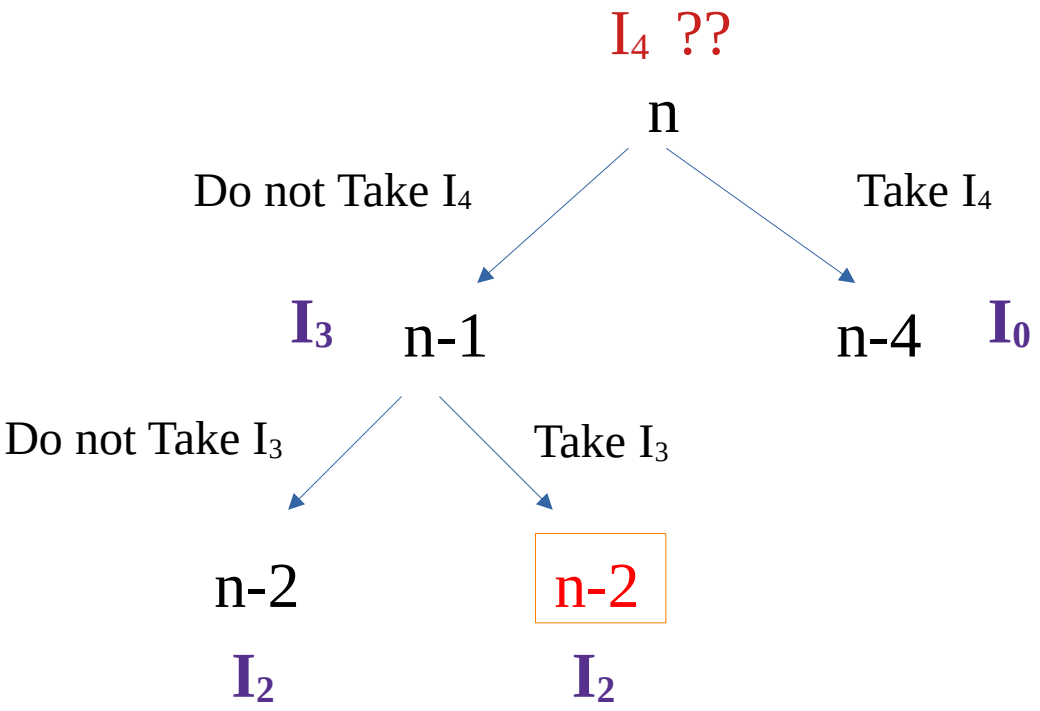
How do you solve the recurrence?

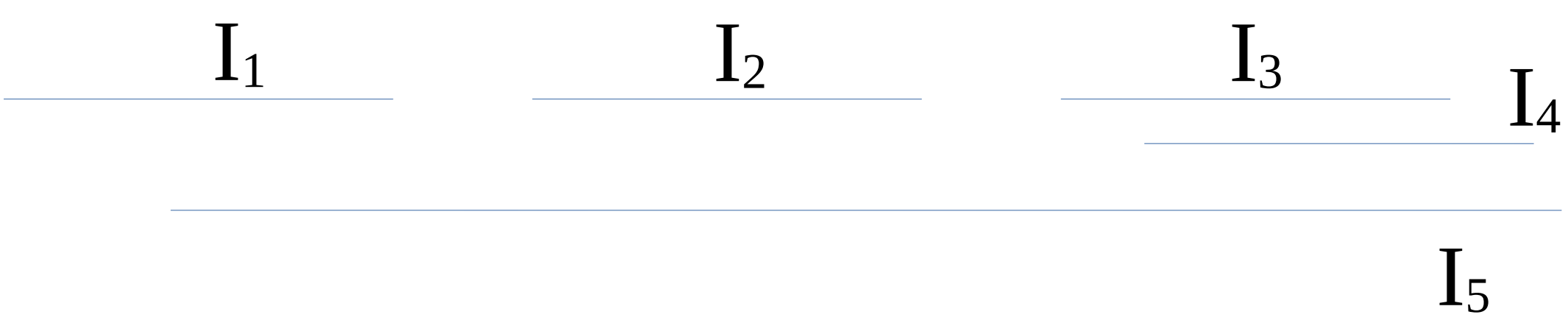
Does the 1.618^n algorithm work for any input?

I_1 I_2 I_3

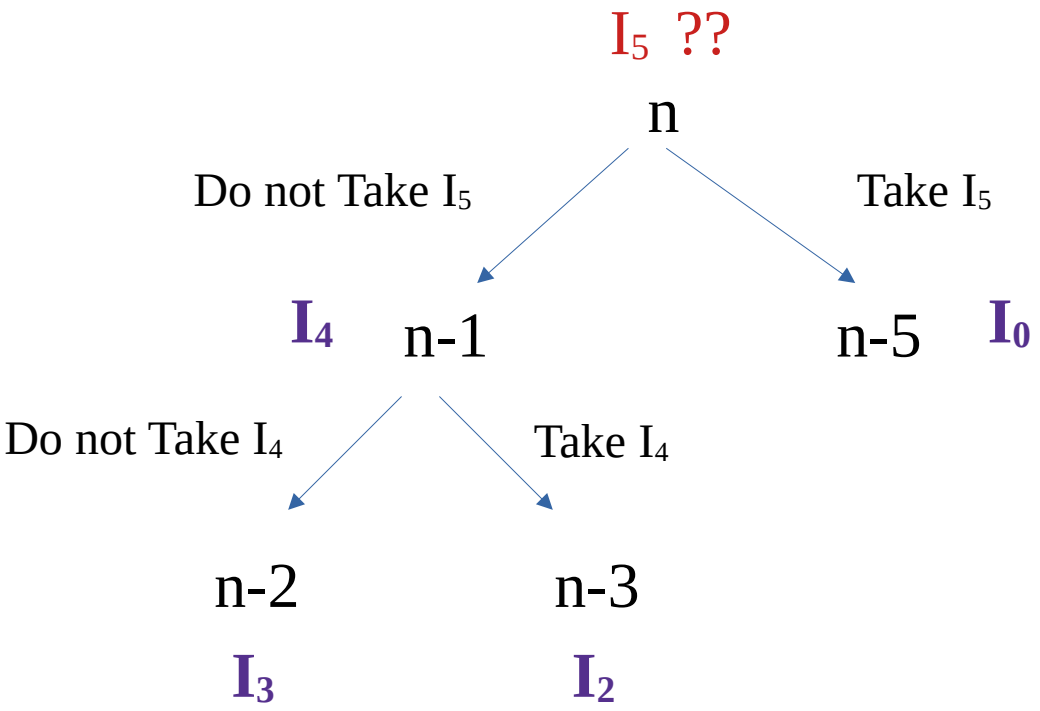
I_4

Interval	Critical Index
I1	0
I2	1
I3	2
I4	0





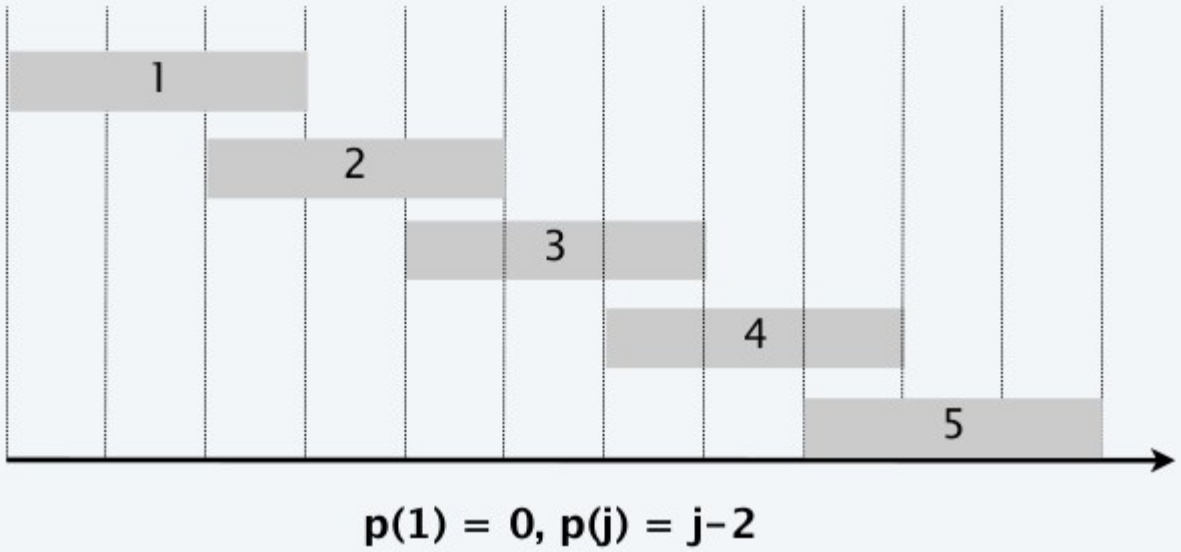
Interval	Critical Index
I_1	0
I_2	1
I_3	2
I_4	2
I_5	0

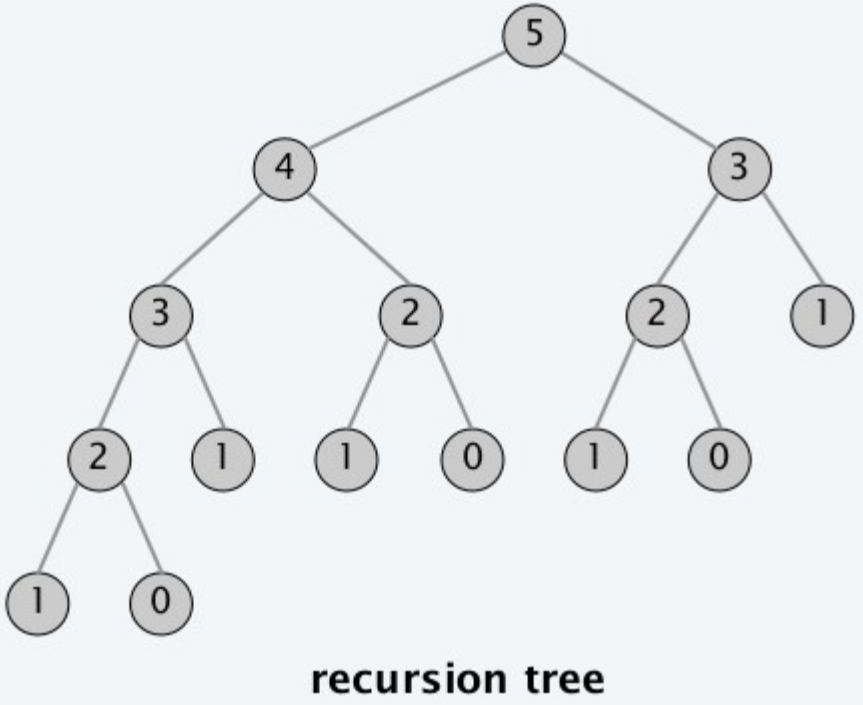


Weighted Interval Scheduling (algorithms)

Approach	Time
Brute Force	$O(2^n \cdot n^c)$
Greedy	Does not work
Recursive algorithm	$O(1.618^n \, n^c)$

Can we do better?





What changes to make?

#recursive calls?

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. Return COMPUTE-OPT(n).

```
COMPUTE-OPT(j)

IF (j = 0)
    RETURN 0.
ELSE
    RETURN max {COMPUTE-OPT(j – 1), w_j + COMPUTE-OPT(p[j]) }.
```

Top-down with memoization

- Solve recursively, but store each result in a table.
- To find the solution to a subproblem, first look in the table. If the answer is there, use it.
- Otherwise, compute the solution to the subproblem and then store the solution in the table for future use.

Memoization: The technique of storing results of expensive function calls

Recursive + Caching

Version – 2

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. M[0]=0
4. Return **M-COMPUTE-OPT(n)**.

Version – 2

#recursive calls?

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. M[0]=0
4. Return **M-COMPUTE-OPT(n)**.

```
M-COMPUTE-OPT(j)

IF (M[j] is uninitialized)
    M[j] ← max { M-COMPUTE-OPT(j – 1), w_j + M-COMPUTE-OPT(p[j]) }.
RETURN M[j].
```

Steps of Dynamic Programming

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from computed information.

Version – 2

What is the solution?

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. M[0]=0
4. Return **M-COMPUTE-OPT(n)**.

```
M-COMPUTE-OPT(j)

IF (M[j] is uninitialized)
    M[j] ← max { M-COMPUTE-OPT(j – 1), w_j + M-COMPUTE-OPT(p[j]) }.
RETURN M[j].
```

Solution – 2

#recursive calls?

1. Go over each of the values once again.

$$M[j] = \max \{ M[j-1], w_j + M[p[j]] \}$$

2. Function SOLUTION(n)

SOLUTION (j)

if ($M[j-1] > w_j + M[p[j]]$)

- Return SOLUTION (j-1)

else

- Return I_j + SOLUTION (p[j])

Version – 3

Bottom-up dynamic programming

Sort the subproblems by size and solve the smaller ones first.

That way, when solving a subproblem, have already solved the smaller subproblems needed.

Iterative

Version – 3

Bottom-up dynamic programming

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. M[0]=0
4. For j=1 to n

$$\mathbf{M[j] = max \{ M[j-1], w_j + M[p[j]] \}}$$

Version – 3

Running time?

Bottom-up dynamic programming

1. Sort the Intervals based on their finish times.
2. Find the critical points for each interval, say p[j] for the interval I_j
3. M[0]=0
4. For j=1 to n

M[j] = max { M[j-1], w_j + M[p[j]] }

Sort the subproblems by size and solve the smaller ones first. That way, when solving a subproblem, have already solved the smaller subproblems needed.

Bottom-up vs Top-down

Top-down: You're solving on demand, and only solving the subproblems you need.

Bottom-up: You compute all subproblems in the correct order, even if some aren't strictly needed.

Tabulation: There's **no need to check and remember** whether a subproblem was already solved — because you're solving them once and in order.

Bottom-up vs Top-down

Feature	Top-down(memoization)	Bottom-up
Uses recursion	Yes	No
Caches subproblems	Only needed ones	All subproblems are computed
Easier to implement?	Often easier	Slightly more code?
Handles large input well?	May hit recursion limit	YES
Control flow	Recursive calls	Nested for-loops

Weighted Interval Scheduling (algorithms)

Approach	Time
Brute Force	$O(2^n \cdot n^c)$
Greedy	Does not work
Recursive algorithm	$O(1.618^n \, n^c)$
Dynamic Programming	O(n log n)